DAT250 Security Remediation and Verification

Assignment 2

Group member: Alex Armand

Student ID: 285230

Group GitHub repository: https://github.com/xeloyzus/UiS-Vulnerable-Student-App

Verification date 8 November 2026

1 Introduction

DAT250 SecureApp is a local FastAPI and PostgreSQL lab that simulates a student portal. This work addressed the seven findings demonstrated in Assignment 1 and verified each fix against the original attack pattern and a legitimate user workflow and implemented the fixes.

The patched build denied cross-account profile and user-management requests, password guessing, rejected unsafe and duplicate uploads, rendered the message payload as text, and kept injected SQL out of query execution.

2 Vulnerability Summary

ID	Finding and category	Affected component and impact
VA-01	Profile IDOR Access control	GET /profile/{user_id} Disclosed another user's profile through a path ID.
VA-02	TA access to user management Access control	GET /admin/users and related actions Exposed administrative data and role controls.
VA-03	Unthrottled login burst Authentication	POST /login Allowed repeated password guesses without a rate limits
VA-04	Unsafe file upload File handling	POST /assignments/upload Accepted and served unexpected active file types.
VA-05	Duplicate submission replay Business logic	POST /assignments/upload An identical request created additional submission rows.
VA-06	Private-message XSS Injection and session protection	POST /messages and recipient views Executed script with access to the session cookie.
VA-07	Course-search SQL injection Injection	GET /courses/search?q=… Allowed unauthorized database reads, including hash exposure in Assignment 1.

3 Root Cause Analysis

VA 01 Profile IDOR

profile_by_id authenticated the session and looked up the given ID directly. Authentication established the caller's identity, but the handler omitted the ownership check before returning personal information.

VA 02 Teaching Assistant Access to User Management

The user list and search required a session and omitted the administrator-role check. The role update accepted a client-supplied X-Role header or an allowed role value as authority. The profile update also wrote a role received from the browser, so a caller could promote their own account.

VA 03 Unthrottled Login Burst

Every POST /login performed an account lookup and credential check without counting attempts. The observed 10-request burst therefore reached authentication in full.

VA 04 Unsafe File Upload

The assignment handler copied the multipart filename and bytes into the shared upload directory without an allowlist or content check. Both Nginx and the backend exposed that directory. Acceptance and direct download were demonstrated in the lab.

VA 05 Duplicate Submission Replay

Every successful upload wrote a file and inserted a row. The handler had no server-side identity for a prior attempt and performed no serialized check before insertion to make sure that submissions are not replayed.

VA 06 Private Message XSS

The inbox, detail and dashboard templates used Jinja's safe filter on message text. That bypassed HTML escaping for content from another user. The login response also issued session_token without HttpOnly, allowing the executed script to read it.

VA 07 Course Search SQL Injection

search_courses interpolated q into a quoted ILIKE expression. A quote in the request could terminate the SQL literal and a UNION clause could return data from another table.

Assessment Boundaries

The seven remediation targets are the demonstrated findings in Assignment 1, sections 3 and 4. The ZAP observations in section 3.1 are tracked separately from this verification. The upload tests demonstrate acceptance and serving of a PHP marker in the original lab. Server-side PHP execution was outside the test.

4 Security Remediation

Python route files below are under backend/app/routes. Template changes are under backend/app/templates.

VA 01 Profile Ownership

profile_routes.py resolves the requested profile from the authenticated session after checking ownership. Personal profile edits still work and the client no longer controls the role column.

```
if user_id != user["id"]:
    raise HTTPException(status_code=403, detail="You can only view your own profile")
```

VA 02 Server Side Roles

admin_routes.py uses the role loaded from the session for the user list, search and role writes. Administrators retain user management. Administrators, teachers and teaching assistants retain grading. Role changes accept only one of the four supported roles.

```
user = authorized_user(request, {"admin"})  # User-management routes
user = authorized_user(request, STAFF_ROLES)  # Grade-management routes
```

VA 03 Login Throttling

auth_routes.py enforces five attempts per normalized account identifier and 30 per source in a 60-second window. It returns 429 with Retry-After before checking credentials. Nginx replaces client-supplied forwarding values before passing the source address to the private backend.

VA 04 Assignment File Validation and Downloads

assignment_routes.py limits files to 10 MiB and checks PDF signatures or the DOCX package structure. The server assigns the storage name. Downloads require ownership and return a nosniff attachment. The public upload-directory mounts were removed. Profile avatars use a separate owner-checked raster-image response.

VA 05 Replay Protection

The upload transaction locks the user and assignment pair before checking for the same filename and content. New paths include a content hash and a random identifier. Older rows can be compared against their existing files. Changed file content can be submitted as a new revision.

```
cur.execute("SELECT pg_advisory_xact_lock(%s, %s)", (user["id"], assignment_id))
```

VA 06 HTML Escaping and Cookie Protection

The message templates now use Jinja's normal HTML escaping. Login sets HttpOnly and SameSite=Lax, and sets Secure when the request is HTTPS. The local lab uses HTTP.

```
<!-- messages.html, inside the inbox loop -->
{{ m.content }}
```

VA 07 Bound Search Parameters

course_routes.py binds q as a parameter while preserving case-insensitive substring search. The user-management search uses the same binding pattern.

```
courses = run_query(
    "SELECT * FROM courses WHERE title ILIKE %s ORDER BY id",
    (f"%{q}%",), fetchall=True,
)
```

5 Verification and Testing

Python Requests and Playwright exercised two loopback-only Docker Compose stacks through Nginx and PostgreSQL. The original stack ran commit c089832 on port 18182. The patched stack ran the working tree on port 18181. The test run passed 37 baseline assertions and 70 patched assertions with disposable lab data.

ID	Before	After and legitimate control
VA-01	Bob changed /profile/5 to /profile/1. Both returned 200 and the latter disclosed the administrator profile.	The same request returned 403 with no target profile data. Bob's own profile remained 200. A signed-out request redirected to login.
VA-02	A TA session opened /admin/users with 200.	TA list, search and forged role update returned 403. A forged profile role left the student role intact. The admin list and valid role changes worked. TA grading still worked and student grade writes returned 403.
VA-03	Ten failed login requests returned 200. No Retry-After header was present.	The same burst returned five 200s followed by five 429s with Retry-After. Correct credentials were accepted after the window. Rotating submitted emails and forwarding headers still hit the source limit.
VA-04	exploit.php returned 302, appeared in submissions and was directly retrievable with 200.	PHP renamed to .pdf returned 415. Oversize returned 413, an unsafe filename 400 and a missing assignment 404. PDF and DOCX uploads worked. The owner downloaded an attachment with identical bytes. Another student and the old direct PHP URL received 404.
VA-05	Replaying the PHP request created two rows. Repeating a valid PDF request also produced two rows.	The PDF returned 302 then 409 and left one row. Concurrent identical requests returned one 302 and one 409. A changed file, another assignment and another student could submit successfully. An identical legacy PDF replay also returned 409.
VA-06	A recipient opening the inbox, message detail or dashboard triggered a browser alert containing the session cookie.	All three views displayed escaped text and triggered zero alerts. The browser could not read session_token. Plain-text messaging and normal login succeeded.

ID	Before	After and legitimate control
VA-07	A UNION probe returned a course-shaped row from users and disclosed the seeded administrator name.	The same probe returned 200 with zero result cards and no seeded name. Searching for DAT250 returned the intended course.

5 Verification and Testing

Reproducible Request Examples

Start the lab with docker compose up --build. Run the commands below in Bash with the documented lab accounts. Set BOB_EMAIL, BOB_PASSWORD, TA_EMAIL and TA_PASSWORD in the shell. Use a disposable database, a new sample.pdf and a fresh login window for the replay and throttle checks.

```
BASE=http://127.0.0.1:18181
curl -sS -c bob.cookies --data-urlencode "email=$BOB_EMAIL" \
  --data-urlencode "password=$BOB_PASSWORD" "$BASE/login"
curl -sS -c ta.cookies --data-urlencode "email=$TA_EMAIL" \
  --data-urlencode "password=$TA_PASSWORD" "$BASE/login"
curl -si -b bob.cookies "$BASE/profile/5"
curl -si -b bob.cookies "$BASE/profile/1"
curl -si -b ta.cookies "$BASE/admin/users"

for i in {1..10}; do
  curl -sS -o /dev/null -w '%{http_code}\n' \
    -d 'email=charlie@student.uis.no&password=invalid' "$BASE/login"
done

printf '%s\n' '<?php /* DAT250 inert lab marker */ ?>' > exploit.php
curl -si -b bob.cookies -F assignment_id=2 \
  -F 'file=@exploit.php;type=application/x-httpd-php' "$BASE/assignments/upload"
for i in 1 2; do
  curl -si -b bob.cookies -F assignment_id=2 \
    -F 'file=@sample.pdf;type=application/pdf' "$BASE/assignments/upload"
done
```

In a separate Alice session, POST recipient_id=5 and the content below to /messages.

As Bob, open the inbox, that message's detail and the dashboard.

Check for visible text, no alert and an HttpOnly session cookie.

Use the UNION input as the q parameter on /courses/search and compare its results with q=DAT250.

```
<script>alert(document.cookie)</script>

nomatch%' UNION ALL SELECT id, 'DAT250_SQL_PROBE', full_name, 0
FROM users WHERE id=1 --
```

6 Lessons Learned

Authorization needs a policy at every entry point that reads or changes privileged data. Testing the profile form exposed a second path to role escalation. Escaping all message views mattered because a single stored value appeared in three different templates.

File validation and replay prevention solve separate problems. The PHP test verified the type boundary, while a valid PDF and simultaneous requests verified the transaction boundary. The login test also demonstrated the importance of checking cooldown expiry and using a trusted source address. The limiter is local to the single backend process used by this lab.